TEST-1

- Start 2024-03-11T15:47:28Z
- End: 2024-03-11T16:06:38Z
- Status: Run completed

Device Identification

Device	TEST-1
Site	
Make	ACME electronics
Model	LINX-215
Software	firmware: 8.2.2, driver: 2.1.0

Summary

	Feature	Stable	Beta	Preview
-	endpoint.config	0/0	0/0	0/6
-	enumeration	0/0	0/0	0/1
-	enumeration.features	0/0	0/0	0/1
✕	gateway	0/0	0/1	0/1
✓	pointset	0/0	4/4	0/0
✓	system	1/1	7/7	0/3

Results

Bucket	Feature	Stage	Result	Descripti
endpoint.config	endpoint_connection_error	preview	fail	Timeout waiting for blobset

Bucket	Feature	Stage	Result	Descripti
				entry conf status is error
endpoint.config	endpoint_connection_retry	preview	fail	Timeout waiting fo blobset entry conf status is error
endpoint.config	endpoint_connection_success_alternate	preview	skip	No functional alternate registry defined
endpoint.config	endpoint_connection_success_reconnect	preview	skip	No functional alternate registry defined
endpoint.config	endpoint_failure_and_restart	preview	fail	Timeout waiting fo blobset entry conf status is error
endpoint.config	endpoint_redirect_and_restart	preview	skip	No functional alternate registry defined
enumeration	empty_enumeration	preview	fail	Timeout waiting fo enumerati not active

Bucket	Feature	Stage	Result	Descripti
enumeration.features	feature_enumeration	preview	fail	Timeout waiting fo enumerati not active
gateway	bad_target_family	preview	skip	Not a proxied device
gateway	gateway_proxy_events	beta	skip	Not a gateway
pointset	pointset_publish	beta	pass	Sequence complete
pointset	pointset_publish_interval	beta	pass	Sequence complete
pointset	pointset_remove_point	beta	pass	Sequence complete
pointset	pointset_request_extraneous	beta	pass	Sequence complete
system	broken_config	beta	pass	Sequence complete
system	config_logging	beta	pass	Sequence complete
system	device_config_acked	beta	pass	Sequence complete
system	extra_config	beta	pass	Sequence complete
system	family_ether_addr	preview	skip	No ether address defined in metadata
system	family_ipv4_addr	preview	skip	No ipv4 address

Bucket	Feature	Stage	Result	Descripti
				defined in metadata
system	family_ipv6_addr	preview	skip	No ipv6 address defined in metadata
system	state_make_model	beta	pass	Sequence complete
system	state_software	beta	pass	Sequence complete
system	system_last_update	stable	pass	Sequence complete
system	valid_serial_no	beta	pass	Sequence complete

Schema

Stage	Result
stable	✓
beta	✓
preview	-

Sequences

endpoint.config

endpoint_connection_error

Description	Push endpoint config message to device that results in a connection error.
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    4. Wait for endpoint config blobset state not defined
```

endpoint_connection_retry

Description	Check repeated endpoint with same information gets retried.
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Update config before blobset entry config status is error:
        * Set `blobset.blobs._iot_endpoint_config.generation` = `new generation`
    4. Wait for blobset entry config status is error
    5. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    6. Wait for endpoint config blobset state not defined
```

endpoint_connection_success_alternate

Description	Check connection to an alternate project.
Result	skip
Summary	No functional alternate registry defined

```
✓   1. Wait for initial last_config matches config timestamp

--------------------------------------------------------------------------------
×   2. Update config mirroring config false:
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
--------------------------------------------------------------------------------

    3. Wait for blobset phase is apply and stateStatus is null
    4. Wait for blobset phase is final and stateStatus is null
    5. Wait for alternate last_config matches config timestamp
    6. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    7. Wait for endpoint config blobset state not defined
    8. Update config mirroring config true:
        * Add `blobset.blobs._iot_endpoint_config` = { "phase": `final`, "generation": `b
    9. Wait for blobset phase is apply and stateStatus is null
    10. Wait for blobset phase is final and stateStatus is null
    11. Wait for restored last_config matches config timestamp
    12. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    13. Wait for endpoint config blobset state not defined
```

endpoint_connection_success_reconnect

Description	Check a successful reconnect to the same endpoint.
Result	skip
Summary	No functional alternate registry defined

```
--------------------------------------------------------------------------------
×   1. Update config before blobset phase is final and stateStatus is null:
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
--------------------------------------------------------------------------------

    2. Wait for blobset phase is final and stateStatus is null
    3. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    4. Wait for endpoint config blobset state not defined
```

endpoint_failure_and_restart

Description	
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Wait for last_start is not zero
    4. Check that initial count is greater than 0
    5. Update config before system mode is ACTIVE:
        * Add `system.operation.mode` = `active`
    6. Wait for system mode is ACTIVE
    7. Update config before system mode is INITIAL:
        * Set `system.operation.mode` = `restart`
    8. Wait for system mode is INITIAL
    9. Check that restart count increased by one
    10. Update config before system mode is ACTIVE:
        * Set `system.operation.mode` = `active`
    11. Wait for system mode is ACTIVE
    12. Wait for last_config is newer than previous last_config before abort
    13. Wait for last_config is newer than previous last_config after abort
    14. Wait for last_start is newer than previous last_start
    15. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    16. Wait for endpoint config blobset state not defined
```

endpoint_redirect_and_restart

Description	
Result	skip
Summary	No functional alternate registry defined

```
✓   1. Wait for initial last_config matches config timestamp

-------------------------------------------------------------------------------------------
×   2. Update config mirroring config false:
```

```
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
---------------------------------------------------------------------------------------------

    3. Wait for blobset phase is apply and stateStatus is null
    4. Wait for blobset phase is final and stateStatus is null
    5. Wait for alternate last_config matches config timestamp
    6. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    7. Wait for endpoint config blobset state not defined
    8. Wait for last_start is not zero
    9. Check that initial count is greater than 0
    10. Update config before system mode is ACTIVE:
        * Add `system.operation.mode` = `active`
    11. Wait for system mode is ACTIVE
    12. Update config before system mode is INITIAL:
        * Set `system.operation.mode` = `restart`
    13. Wait for system mode is INITIAL
    14. Check that restart count increased by one
    15. Update config before system mode is ACTIVE:
        * Set `system.operation.mode` = `active`
    16. Wait for system mode is ACTIVE
    17. Wait for last_config is newer than previous last_config before abort
    18. Wait for last_config is newer than previous last_config after abort
    19. Wait for last_start is newer than previous last_start
    20. Update config mirroring config true:
        * Add `blobset.blobs._iot_endpoint_config` = { "phase": `final`, "generation": `b
    21. Wait for blobset phase is apply and stateStatus is null
    22. Wait for blobset phase is final and stateStatus is null
    23. Wait for restored last_config matches config timestamp
    24. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    25. Wait for endpoint config blobset state not defined
```

enumeration

empty_enumeration

Description	Check enumeration of nothing at all
Result	fail
Summary	Timeout waiting for enumeration not active

```
✓   1. Update config before enumeration not active:
        * Add `discovery` = { "enumerate": {  } }
```

```
  ----------------------------------------
  ×   2. Wait for enumeration not active ×
  ----------------------------------------

      3. Update config before matching enumeration generation:
          * Add `discovery.generation` = `generation start time`
      4. Wait for matching enumeration generation
      5. Update config before cleared enumeration generation:
          * Remove `discovery.generation`
      6. Wait for cleared enumeration generation
      7. Check that no family enumeration
      8. Check that no feature enumeration
      9. Check that no point enumeration
```

enumeration.features

feature_enumeration

Description	Check enumeration of device features
Result	fail
Summary	Timeout waiting for enumeration not active

```
  ✓   1. Update config before enumeration not active:
          * Add `discovery` = { "enumerate": { "features": `true` } }

  ----------------------------------------
  ×   2. Wait for enumeration not active ×
  ----------------------------------------

      3. Update config before matching enumeration generation:
          * Add `discovery.generation` = `generation start time`
      4. Wait for matching enumeration generation
      5. Update config before cleared enumeration generation:
          * Remove `discovery.generation`
      6. Wait for cleared enumeration generation
      7. Check that no family enumeration
      8. Check that feature enumeration matches metadata
      9. Check that all enumerated features are official buckets
      10. Check that no point enumeration
```

gateway

bad_target_family

Description	Error handling for badly formed target address family
Result	skip
Summary	Not a proxied device

```
-------------------------------------------
×   1. Test skipped: Not a proxied device ×
-------------------------------------------
```

gateway_proxy_events

Description	Check that a gateway proxies pointset events for indicated devices
Result	skip
Summary	Not a gateway

```
-------------------------------------
×   1. Test skipped: Not a gateway ×
-------------------------------------
```

pointset

pointset_publish

Description	Check that a device publishes pointset events
Result	pass
Summary	Sequence complete

```
✓   1. Wait for receive a pointset event
```

pointset_publish_interval

Description	Check handling of sample_rate_sec and sample_limit_sec
Result	pass
Summary	Sequence complete

```
✓   1. Update config before receive at least 4 pointset events:
        * Set `pointset.sample_rate_sec` = `8`
✓   2. Wait for receive at least 4 pointset events
✓   3. Check that time period between successive pointset events is between 5 and 8 secon
✓   4. Update config before receive at least 4 pointset events:
        * Set `pointset.sample_rate_sec` = `18`
        * Set `pointset.sample_limit_sec` = `15`
✓   5. Wait for receive at least 4 pointset events
✓   6. Check that time period between successive pointset events is between 15 and 18 sec
```

pointset_remove_point

Description	Check that pointset state does not report an unconfigured point
Result	pass
Summary	Sequence complete

```
✓   1. Wait for pointset state matches config
✓   2. Wait for pointset event contains correct points
✓   3. Update config before pointset status does not contain removed point:
        * Remove `pointset.points[random_point]`
✓   4. Wait for pointset status does not contain removed point
✓   5. Wait for pointset state matches config
✓   6. Wait for pointset event contains correct points
✓   7. Update config before pointset status contains removed point:
        * Add `pointset.points[random_point]` = point configuration
✓   8. Wait for pointset status contains removed point
✓   9. Wait for pointset state matches config
✓   10. Wait for pointset event contains correct points
```

pointset_request_extraneous

Description	Check error when pointset configuration contains extraneous point
Result	pass
Summary	Sequence complete

```
✓   1. Update config Before pointset state matches config:
        * Set `pointset.sample_rate_sec` = `10`
✓   2. Wait for pointset state matches config
✓   3. Wait for pointset event contains correct points
✓   4. Update config before pointset status contains extraneous point error:
        * Add `pointset.points[extraneous_point]` = point configuration
✓   5. Wait for pointset status contains extraneous point error
✓   6. Wait for pointset state matches config
✓   7. Wait for pointset event contains correct points
✓   8. Update config before pointset status removes extraneous point error:
        * Remove `pointset.points[extraneous_point]`
✓   9. Wait for pointset status removes extraneous point error
✓   10. Wait for pointset state matches config
✓   11. Wait for pointset event contains correct points
```

system

broken_config

Description	Check that the device correctly handles a broken (non-json) config message.
Result	pass
Summary	Sequence complete

```
✓   1. Update config starting broken_config:
        * Set `system.min_loglevel` = `100`
✓   2. Wait for initial state synchronized
✓   3. Check that initial stable_config matches last_config
✓   4. Wait for log category `system.config.apply` level `NOTICE` to be logged
✓   5. Wait for has applicable system status
✓   6. Check that applicable system status
✓   7. Force reset config
✓   8. Wait for state last_config sync
✓   9. Update config before last_config updated:
        * Set `system.min_loglevel` = `100`
✓   10. Wait for last_config updated
```

```
----------------------------------------
×   11. Wait for state last_config sync ×
----------------------------------------

    12. Wait for log category `system.config.apply` level `NOTICE` to be logged
    13. Wait for restored state synchronized
    14. Update config before last_config updated:
        * Set `system.min_loglevel` = `100`
    15. Wait for last_config updated
    16. Wait for log category `system.config.apply` level `NOTICE` to be logged
    17. Check that log category `system.config.receive` level `DEBUG` not logged
    18. Check that log category `system.config.parse` level `DEBUG` not logged
```

config_logging

Description	Check that the device publishes minimum required log entries when receiving config
Result	pass
Summary	Sequence complete

```
✓   1. Update config set min_loglevel to debug:
        * Set `system.min_loglevel` = `100`
✓   2. Force config update to resend config to device
✓   3. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   4. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   5. Wait for log category `system.config.apply` level `NOTICE` to be logged
```

device_config_acked

Description	Check that the device MQTT-acknowledges a sent config.
Result	pass
Summary	Sequence complete

```
✓   1. Wait for config acked
```

extra_config

Description	Check that the device correctly handles an extra out-of-schema field

Result	pass
Summary	Sequence complete

```
✓   1. Update config before last_config not null:
        * Set `system.min_loglevel` = `100`
✓   2. Wait for last_config not null
✓   3. Wait for system operational
✓   4. Check that no applicable system status
✓   5. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   6. Wait for last_config updated
✓   7. Wait for system operational
✓   8. Check that no applicable system status
✓   9. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   10. Wait for log category `system.config.apply` level `NOTICE` to be logged
✓   11. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   12. Wait for last_config updated again
✓   13. Wait for system operational
✓   14. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   15. Wait for log category `system.config.apply` level `NOTICE` to be logged
```

family_ether_addr

Description	
Result	skip
Summary	No ether address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ether address matches
```

family_ipv4_addr

Description	
Result	skip

Summary	No ipv4 address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ipv4 address matches
```

family_ipv6_addr

Description	
Result	skip
Summary	No ipv6 address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ipv6 address matches
```

state_make_model

Description	Check that a device publishes correct make and model information in state messages
Result	pass
Summary	Sequence complete

```
✓   1. Check that make and model in state matches make in metadata
```

state_software

Description	Check that a device publishes correct software information in state messages
Result	pass

Summary	Sequence complete

```
✓   1. Check that software in metadata matches state
```

system_last_update

Description	Check that last_update state is correctly set in response to a config update.
Result	pass
Summary	Sequence complete

```
✓   1. Wait for state last_config matches first config timestamp
✓   2. Wait for state update complete
✓   3. Force config update to trigger another config update
✓   4. Wait for state last_config matches new config timestamp
✓   5. Wait for state update complete
✓   6. Force config update to trigger another config update
✓   7. Wait for state last_config matches last config timestamp
✓   8. Wait for state update complete
```

valid_serial_no

Description	
Result	pass
Summary	Sequence complete

```
✓   1. Wait for received serial number matches
```

TEST-1

- Start 2024-03-11T15:47:28Z
- End: 2024-03-11T16:06:38Z
- Status: Run completed

Device Identification

Device	TEST-1
Site	
Make	ACME electronics
Model	LINX-215
Software	firmware: 8.2.2, driver: 2.1.0

Summary

	Feature	Stable	Beta	Preview
-	endpoint.config	0/0	0/0	0/6
-	enumeration	0/0	0/0	0/1
-	enumeration.features	0/0	0/0	0/1
✕	gateway	0/0	0/1	0/1
✓	pointset	0/0	4/4	0/0
✓	system	1/1	7/7	0/3

Results

Bucket	Feature	Stage	Result	Descripti
endpoint.config	endpoint_connection_error	preview	fail	Timeout waiting fo blobset entry conf status is error
endpoint.config	endpoint_connection_retry	preview	fail	Timeout waiting fo blobset entry conf

Bucket	Feature	Stage	Result	Descripti
				status is error
endpoint.config	endpoint_connection_success_alternate	preview	skip	No functional alternate registry defined
endpoint.config	endpoint_connection_success_reconnect	preview	skip	No functional alternate registry defined
endpoint.config	endpoint_failure_and_restart	preview	fail	Timeout waiting for blobset entry conf status is error
endpoint.config	endpoint_redirect_and_restart	preview	skip	No functional alternate registry defined
enumeration	empty_enumeration	preview	fail	Timeout waiting for enumerati not active
enumeration.features	feature_enumeration	preview	fail	Timeout waiting for enumerati not active
gateway	bad_target_family	preview	skip	Not a proxied device

Bucket	Feature	Stage	Result	Descripti
gateway	gateway_proxy_events	beta	skip	Not a gateway
pointset	pointset_publish	beta	pass	Sequence complete
pointset	pointset_publish_interval	beta	pass	Sequence complete
pointset	pointset_remove_point	beta	pass	Sequence complete
pointset	pointset_request_extraneous	beta	pass	Sequence complete
system	broken_config	beta	pass	Sequence complete
system	config_logging	beta	pass	Sequence complete
system	device_config_acked	beta	pass	Sequence complete
system	extra_config	beta	pass	Sequence complete
system	family_ether_addr	preview	skip	No ether address defined in metadata
system	family_ipv4_addr	preview	skip	No ipv4 address defined in metadata
system	family_ipv6_addr	preview	skip	No ipv6 address defined in metadata
system	state_make_model	beta	pass	Sequence complete

Bucket	Feature	Stage	Result	Descripti
system	state_software	beta	pass	Sequence complete
system	system_last_update	stable	pass	Sequence complete
system	valid_serial_no	beta	pass	Sequence complete

Schema

Stage	Result
stable	✓
beta	✓
preview	-

Sequences

endpoint.config

endpoint_connection_error

Description	Push endpoint config message to device that results in a connection error.
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Update config before endpoint config blobset state not defined:
```

```
        * Remove `blobset.blobs._iot_endpoint_config`
    4. Wait for endpoint config blobset state not defined
```

endpoint_connection_retry

Description	Check repeated endpoint with same information gets retried.
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Update config before blobset entry config status is error:
        * Set `blobset.blobs._iot_endpoint_config.generation` = `new generation`
    4. Wait for blobset entry config status is error
    5. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    6. Wait for endpoint config blobset state not defined
```

endpoint_connection_success_alternate

Description	Check connection to an alternate project.
Result	skip
Summary	No functional alternate registry defined

```
✓   1. Wait for initial last_config matches config timestamp

-------------------------------------------------------------------------------------------
×   2. Update config mirroring config false:
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
-------------------------------------------------------------------------------------------

    3. Wait for blobset phase is apply and stateStatus is null
    4. Wait for blobset phase is final and stateStatus is null
    5. Wait for alternate last_config matches config timestamp
```

```
    6. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    7. Wait for endpoint config blobset state not defined
    8. Update config mirroring config true:
        * Add `blobset.blobs._iot_endpoint_config` = { "phase": `final`, "generation": `b
    9. Wait for blobset phase is apply and stateStatus is null
    10. Wait for blobset phase is final and stateStatus is null
    11. Wait for restored last_config matches config timestamp
    12. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    13. Wait for endpoint config blobset state not defined
```

endpoint_connection_success_reconnect

Description	Check a successful reconnect to the same endpoint.
Result	skip
Summary	No functional alternate registry defined

```
---------------------------------------------------------------------------------------
×   1. Update config before blobset phase is final and stateStatus is null:
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
---------------------------------------------------------------------------------------

    2. Wait for blobset phase is final and stateStatus is null
    3. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    4. Wait for endpoint config blobset state not defined
```

endpoint_failure_and_restart

Description	
Result	fail
Summary	Timeout waiting for blobset entry config status is error

```
✓   1. Update config before blobset entry config status is error:
        * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener

-------------------------------------------------------
```

```
×   2. Wait for blobset entry config status is error ×
-------------------------------------------------------

    3. Wait for last_start is not zero
    4. Check that initial count is greater than 0
    5. Update config before system mode is ACTIVE:
        * Add `system.operation.mode` = `active`
    6. Wait for system mode is ACTIVE
    7. Update config before system mode is INITIAL:
        * Set `system.operation.mode` = `restart`
    8. Wait for system mode is INITIAL
    9. Check that restart count increased by one
    10. Update config before system mode is ACTIVE:
        * Set `system.operation.mode` = `active`
    11. Wait for system mode is ACTIVE
    12. Wait for last_config is newer than previous last_config before abort
    13. Wait for last_config is newer than previous last_config after abort
    14. Wait for last_start is newer than previous last_start
    15. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    16. Wait for endpoint config blobset state not defined
```

endpoint_redirect_and_restart

Description	
Result	skip
Summary	No functional alternate registry defined

```
✓   1. Wait for initial last_config matches config timestamp

-------------------------------------------------------------------------------------------
×   2. Update config mirroring config false:
×       * Add `blobset` = { "blobs": { "_iot_endpoint_config": { "phase": `final`, "gener
-------------------------------------------------------------------------------------------

    3. Wait for blobset phase is apply and stateStatus is null
    4. Wait for blobset phase is final and stateStatus is null
    5. Wait for alternate last_config matches config timestamp
    6. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    7. Wait for endpoint config blobset state not defined
    8. Wait for last_start is not zero
    9. Check that initial count is greater than 0
    10. Update config before system mode is ACTIVE:
```

```
        * Add `system.operation.mode` = `active`
    11. Wait for system mode is ACTIVE
    12. Update config before system mode is INITIAL:
        * Set `system.operation.mode` = `restart`
    13. Wait for system mode is INITIAL
    14. Check that restart count increased by one
    15. Update config before system mode is ACTIVE:
        * Set `system.operation.mode` = `active`
    16. Wait for system mode is ACTIVE
    17. Wait for last_config is newer than previous last_config before abort
    18. Wait for last_config is newer than previous last_config after abort
    19. Wait for last_start is newer than previous last_start
    20. Update config mirroring config true:
        * Add `blobset.blobs._iot_endpoint_config` = { "phase": `final`, "generation": `b
    21. Wait for blobset phase is apply and stateStatus is null
    22. Wait for blobset phase is final and stateStatus is null
    23. Wait for restored last_config matches config timestamp
    24. Update config before endpoint config blobset state not defined:
        * Remove `blobset.blobs._iot_endpoint_config`
    25. Wait for endpoint config blobset state not defined
```

enumeration

empty_enumeration

Description	Check enumeration of nothing at all
Result	fail
Summary	Timeout waiting for enumeration not active

```
✓   1. Update config before enumeration not active:
        * Add `discovery` = { "enumerate": {  } }

----------------------------------------
×   2. Wait for enumeration not active ×
----------------------------------------

    3. Update config before matching enumeration generation:
        * Add `discovery.generation` = `generation start time`
    4. Wait for matching enumeration generation
    5. Update config before cleared enumeration generation:
        * Remove `discovery.generation`
    6. Wait for cleared enumeration generation
    7. Check that no family enumeration
```

```
    8. Check that no feature enumeration
    9. Check that no point enumeration
```

enumeration.features

feature_enumeration

Description	Check enumeration of device features
Result	fail
Summary	Timeout waiting for enumeration not active

```
✓   1. Update config before enumeration not active:
        * Add `discovery` = { "enumerate": { "features": `true` } }

----------------------------------------
×   2. Wait for enumeration not active ×
----------------------------------------

    3. Update config before matching enumeration generation:
        * Add `discovery.generation` = `generation start time`
    4. Wait for matching enumeration generation
    5. Update config before cleared enumeration generation:
        * Remove `discovery.generation`
    6. Wait for cleared enumeration generation
    7. Check that no family enumeration
    8. Check that feature enumeration matches metadata
    9. Check that all enumerated features are official buckets
    10. Check that no point enumeration
```

gateway

bad_target_family

Description	Error handling for badly formed target address family
Result	skip
Summary	Not a proxied device

```
-------------------------------------------
```

```
×   1. Test skipped: Not a proxied device ×
-------------------------------------------
```

gateway_proxy_events

Description	Check that a gateway proxies pointset events for indicated devices
Result	skip
Summary	Not a gateway

```
-------------------------------------
×   1. Test skipped: Not a gateway ×
-------------------------------------
```

pointset

pointset_publish

Description	Check that a device publishes pointset events
Result	pass
Summary	Sequence complete

```
✓   1. Wait for receive a pointset event
```

pointset_publish_interval

Description	Check handling of sample_rate_sec and sample_limit_sec
Result	pass
Summary	Sequence complete

```
✓   1. Update config before receive at least 4 pointset events:
        * Set `pointset.sample_rate_sec` = `8`
```

```
✓   2. Wait for receive at least 4 pointset events
✓   3. Check that time period between successive pointset events is between 5 and 8 secon
✓   4. Update config before receive at least 4 pointset events:
        * Set `pointset.sample_rate_sec` = `18`
        * Set `pointset.sample_limit_sec` = `15`
✓   5. Wait for receive at least 4 pointset events
✓   6. Check that time period between successive pointset events is between 15 and 18 sec
```

pointset_remove_point

Description	Check that pointset state does not report an unconfigured point
Result	pass
Summary	Sequence complete

```
✓   1. Wait for pointset state matches config
✓   2. Wait for pointset event contains correct points
✓   3. Update config before pointset status does not contain removed point:
        * Remove `pointset.points[random_point]`
✓   4. Wait for pointset status does not contain removed point
✓   5. Wait for pointset state matches config
✓   6. Wait for pointset event contains correct points
✓   7. Update config before pointset status contains removed point:
        * Add `pointset.points[random_point]` = point configuration
✓   8. Wait for pointset status contains removed point
✓   9. Wait for pointset state matches config
✓   10. Wait for pointset event contains correct points
```

pointset_request_extraneous

Description	Check error when pointset configuration contains extraneous point
Result	pass
Summary	Sequence complete

```
✓   1. Update config Before pointset state matches config:
        * Set `pointset.sample_rate_sec` = `10`
✓   2. Wait for pointset state matches config
✓   3. Wait for pointset event contains correct points
✓   4. Update config before pointset status contains extraneous point error:
        * Add `pointset.points[extraneous_point]` = point configuration
✓   5. Wait for pointset status contains extraneous point error
```

```
✓   6. Wait for pointset state matches config
✓   7. Wait for pointset event contains correct points
✓   8. Update config before pointset status removes extraneous point error:
        * Remove `pointset.points[extraneous_point]`
✓   9. Wait for pointset status removes extraneous point error
✓   10. Wait for pointset state matches config
✓   11. Wait for pointset event contains correct points
```

system

broken_config

Description	Check that the device correctly handles a broken (non-json) config message.
Result	pass
Summary	Sequence complete

```
✓   1. Update config starting broken_config:
        * Set `system.min_loglevel` = `100`
✓   2. Wait for initial state synchronized
✓   3. Check that initial stable_config matches last_config
✓   4. Wait for log category `system.config.apply` level `NOTICE` to be logged
✓   5. Wait for has applicable system status
✓   6. Check that applicable system status
✓   7. Force reset config
✓   8. Wait for state last_config sync
✓   9. Update config before last_config updated:
        * Set `system.min_loglevel` = `100`
✓   10. Wait for last_config updated

----------------------------------------
×   11. Wait for state last_config sync ×
----------------------------------------

    12. Wait for log category `system.config.apply` level `NOTICE` to be logged
    13. Wait for restored state synchronized
    14. Update config before last_config updated:
        * Set `system.min_loglevel` = `100`
    15. Wait for last_config updated
    16. Wait for log category `system.config.apply` level `NOTICE` to be logged
    17. Check that log category `system.config.receive` level `DEBUG` not logged
    18. Check that log category `system.config.parse` level `DEBUG` not logged
```

config_logging

Description	Check that the device publishes minimum required log entries when receiving config
Result	pass
Summary	Sequence complete

```
✓   1. Update config set min_loglevel to debug:
        * Set `system.min_loglevel` = `100`
✓   2. Force config update to resend config to device
✓   3. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   4. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   5. Wait for log category `system.config.apply` level `NOTICE` to be logged
```

device_config_acked

Description	Check that the device MQTT-acknowledges a sent config.
Result	pass
Summary	Sequence complete

```
✓   1. Wait for config acked
```

extra_config

Description	Check that the device correctly handles an extra out-of-schema field
Result	pass
Summary	Sequence complete

```
✓   1. Update config before last_config not null:
        * Set `system.min_loglevel` = `100`
✓   2. Wait for last_config not null
✓   3. Wait for system operational
✓   4. Check that no applicable system status
✓   5. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   6. Wait for last_config updated
✓   7. Wait for system operational
✓   8. Check that no applicable system status
✓   9. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   10. Wait for log category `system.config.apply` level `NOTICE` to be logged
```

```
✓   11. Wait for log category `system.config.receive` level `DEBUG` to be logged
✓   12. Wait for last_config updated again
✓   13. Wait for system operational
✓   14. Wait for log category `system.config.parse` level `DEBUG` to be logged
✓   15. Wait for log category `system.config.apply` level `NOTICE` to be logged
```

family_ether_addr

Description	
Result	skip
Summary	No ether address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ether address matches
```

family_ipv4_addr

Description	
Result	skip
Summary	No ipv4 address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ipv4 address matches
```

family_ipv6_addr

Description	
Result	skip

Summary	No ipv6 address defined in metadata

```
---------------------------------------------
×   1. Wait for localnet families available ×
---------------------------------------------

    2. Check that device family ipv6 address matches
```

state_make_model

Description	Check that a device publishes correct make and model information in state messages
Result	pass
Summary	Sequence complete

```
✓   1. Check that make and model in state matches make in metadata
```

state_software

Description	Check that a device publishes correct software information in state messages
Result	pass
Summary	Sequence complete

```
✓   1. Check that software in metadata matches state
```

system_last_update

Description	Check that last_update state is correctly set in response to a config update.
Result	pass
Summary	Sequence complete

```
✓   1. Wait for state last_config matches first config timestamp
✓   2. Wait for state update complete
```

```
✓   3. Force config update to trigger another config update
✓   4. Wait for state last_config matches new config timestamp
✓   5. Wait for state update complete
✓   6. Force config update to trigger another config update
✓   7. Wait for state last_config matches last config timestamp
✓   8. Wait for state update complete
```

valid_serial_no

Description	
Result	pass
Summary	Sequence complete

```
✓   1. Wait for received serial number matches
```